HOSTEL / PG MANAGEMENT SYSTEM

Room Discovery, Booking, Tenancy, Rent & Complaint Management Platform: Complete Project Documentation

Version 1.1 (supersedes v1.0). Items changed in v1.1 are tagged **[v1.1]**.

Change Summary (v1.0 → v1.1)

Area	Change
Schema	`payments.tenancy_id` nullable + CHECK; unique `gateway_order_id`; `webhook_events`; unique `rent_dues(tenancy_id, month)`; unique `(property_id, room_number)`; separate owner-response and payment clocks on bookings; `users.user_id` tied to `auth.users`; DOB/minor/guardian-consent fields
New tables	`property_staff`, `colleges`, `refunds`, `deposit_ledger`, `webhook_events`
Availability	Bookings/tenancies are authoritative; `beds.status` is a trigger-synced cache; partial unique indexes define "active"
Authorization	FastAPI sessions carry the caller's JWT so RLS applies; service-role key only for jobs and webhook
Payments	v1 is record-only (offline advance/rent logged by owner); Razorpay deferred; MVP booking variant added
Rules	Proration, notice, and deposit-settlement rules; money and timezone standards
Compliance	Minor/guardian consent; PII-safe audit log; ID-proof handling; pre-approval owner visibility; gender options
Text fixes	API typo, Section 27 decisions, duplicate "Merge" deferred, phasing and tests updated

1. Executive Summary

The Hostel/PG Management System (HPMS) is a web platform that connects students looking for accommodation with hostel/PG owners. Students register, search available rooms, compare rent and facilities, and book a room. Owners add and manage properties, control room availability, maintain tenant records, track monthly rent and payments, and resolve complaints and maintenance requests, all from one system.

Two portals and one panel share one backend:

- **Student Portal**: registration, search, booking, rent payment records, complaints.
- **Owner Portal**: properties, rooms, tenants, rent ledger, maintenance.
- **Admin Panel**: platform-level verification of owners/properties, dispute handling, reports.

Key principle: the backend is the source of truth for room availability. A room can never be double-booked; booking is an atomic, transactional operation enforced at the database level. **[v1.1]**

2. Goals & Non-Goals

Goals

- One place for students to discover, compare and book verified rooms.
- Give owners full control of properties, occupancy, rent tracking and maintenance.
- Prevent double-booking and lost rent records through transactional integrity.
- Full auditability of bookings, payments, and complaint actions.
- Reports and CSV/Excel export for owners and admin.

Non-Goals (v1)

- No in-app chat/messaging between student and owner (contact details shared after booking request approval only).
- No automatic rent-agreement legal e-signing (PDF agreement generation only).
- No dynamic/AI pricing.
- No mess/food billing module, no biometric/IoT gate access.
- No native mobile app (responsive web only).
- **[v1.1]** No on-platform collection or payout of rent in the first release: rent and advances are recorded by owners (record-only). Online payments (Razorpay) follow in a later phase.

3. System Actors & Roles

Actor	Access	Auth Mechanism
Student	Search, book, view/pay rent, raise complaints, view own records	Supabase Auth (email + password / Google), email verification
Owner	Own properties/rooms/tenants/rent/complaints only	Supabase Auth + email/phone verification; admin

		approval before listings go live
Admin	Full platform access: verify owners/properties, users, reports, audit logs	Supabase Auth + mandatory MFA
Warden/Manager (optional)	Restricted to **assigned property only** (via `property_staff`): attendance, complaints, tenant list	Owner-issued invite; no rent/financial settings

Student: Detailed Spec

- Self-signup with email verification; profile: name, phone, **date of birth**, college/organisation, gender, guardian contact, ID proof (uploaded, private). **[v1.1]**
- Can view only own bookings, payments, complaints, and the tenancy they belong to.
- **[v1.1]** Students under 18 need verified guardian consent before any booking (see Section 16).

Owner: Detailed Spec

- Self-signup → submits KYC (ID proof, property ownership/authorisation doc) → Admin approves → can publish listings.
- Sees only properties they own; cannot see other owners' data.
- **[v1.1] Applicant visibility before approval:** name, gender, college, course, move-in date and duration only. Phone, email, guardian contact and ID proof unlock after the booking is approved, and only for that student.
- Can invite a Warden/Manager scoped to one property (`property_staff`).

Admin: Detailed Spec

- Approves/rejects owners and properties, suspends listings/accounts, resolves disputes, views platform analytics and audit logs.
- All actions by every role are recorded in `audit_logs` with role tag.

4. High-Level Architecture

Surfaces: public website (search), Student Portal, Owner Portal, Admin Panel. All are served by a FastAPI backend, the only component that talks to external services (payment gateway, email/SMS, maps) and holds credentials.

Key Architectural Principles

- Frontend never holds secrets or talks to payment/email providers directly; backend-mediated only.
- Supabase (PostgreSQL) is the single source of truth.
- Booking and payment updates are transactional (row-level locking on the bed, plus a partial unique index as backstop): no double-booking.
- Payment confirmation (when online payments are enabled) is trusted only via verified gateway webhooks, never the client redirect.
- Receipt/agreement PDF generation and email/SMS dispatch are async, retryable, logged jobs.
- All notification messages come from versioned templates.
- Two isolated environments: staging and production.

Data & Integration Flow

- Student/Owner web apps → FastAPI → Supabase (DB, Storage, Auth).
- FastAPI → Payment Gateway (Razorpay, later phase) → webhook back to FastAPI.
- FastAPI → Email (SMTP/SendGrid) and optional SMS/WhatsApp provider.
- FastAPI → Maps API (location/geocoding, distance to college).
- Celery + Redis run scheduled jobs: monthly rent generation, due reminders, overdue flags, booking expiry, retention purge.

4A. Architecture Decisions [v1.1]

Topic	Decision
Authorization path	The frontend never queries the DB directly. All reads/writes go through FastAPI. FastAPI opens each DB session with the caller's JWT claims (`SET LOCAL role authenticated; SET LOCAL request.jwt.claims = ...`) so **RLS applies**. The service-role key is used only by Celery jobs and the payment webhook.
Source of truth for availability	`bookings` and `tenancies`. `beds.status` is a derived cache updated by trigger in the same transaction.
Payment model (v1)	**Record-only.** Owner collects rent/advance directly and logs it. Razorpay is a later phase.
Money	`numeric(12,2)`, INR.
Time	`timestamptz` everywhere. Due dates and day boundaries computed in `Asia/Kolkata`.
Duplicate handling	v1 is flag-only. "Merge" is deferred.

5. Tech Stack (Recommended)

Layer	Technology
Frontend	React (Vite) + Tailwind CSS
Backend	FastAPI (Python)
Database	Supabase (PostgreSQL), separate staging/production projects
File Storage	Supabase Storage (room photos public; ID proofs, agreements, receipts private/signed URLs)
Auth	Supabase Auth / JWT; MFA (TOTP) for Admin
Payments	Razorpay (UPI, cards, netbanking) with webhook verification, **later phase**
Email / SMS	SMTP / SendGrid; optional MSG91/Twilio for SMS
PDF Generation	WeasyPrint / Playwright (rent receipts, rental agreement)
Search	PostgreSQL full-text + filters; **[v1.1]** distance via haversine or `earthdistance` on lat/long against a `colleges` table (upgrade path: PostGIS)
Maps	Google Maps / Mapbox
Background Jobs	Celery + Redis
Bot/Spam Protection	Cloudflare Turnstile / reCAPTCHA v3 + backend rate limiting
Monitoring	Sentry + uptime monitor
CI/CD	GitHub Actions: lint/test on PR, staging on merge to `develop`, prod on tagged release

6. User Onboarding & Verification

6.1 Student Registration

- Form: name, email, phone, **DOB**, gender (male / female / other / prefer not to say), college, guardian contact; consent checkbox with versioned text (`consent_version`, `consented_at`). **[v1.1]**
- CAPTCHA + rate limiting; email verification required before booking. Password reset and phone OTP flows are required. **[v1.1]**
- ID proof upload (optional at signup, mandatory before booking confirmation). Only the image is stored; the ID number is never stored or logged. **[v1.1]**
- **[v1.1]** If the student is under 18, booking is blocked until guardian consent is recorded (guardian contact OTP-verified, versioned consent text).

6.2 Owner Registration & Approval

- Owner submits KYC and property proof → status *Pending Verification* → Admin *Approve / Reject (with reason)*.
- Listings stay hidden until owner AND property are approved.

6.3 Duplicate Detection

- Runs on new users and new properties: normalised email/phone, fuzzy name, and property address + coordinates match.
- On match: flagged; Admin resolves via **Keep Separate** / **Mark Duplicate**. No automatic deletion. **[v1.1]** "Merge" is deferred past v1.

7. Core Modules

Student & Owner registration, Property Management, Room/Bed Management, Search & Filters, Booking Module, Tenant Management, Rent & Payment Module, Complaint & Maintenance Module, Notices & Announcements, Reviews & Ratings, Notifications, Dashboard & Analytics, Reports/Export, Audit Log.

7A. Property Management (Owner)

- Add property: name, type (Boys Hostel / Girls Hostel / Co-ed / PG), address, map pin, nearby landmarks/colleges, description, house rules, photos, contact person.
- Facilities (checkbox set): WiFi, AC, food/mess, laundry, parking, geyser, CCTV, power backup, gym, housekeeping, RO water.
- Policies: security deposit, notice period, lock-in period, visitor timing, gate-closing time.
- Actions: edit, deactivate, delete (blocked if active tenants exist).

7B. Room & Bed Management

- Room: room number (**unique per property [v1.1]**), floor, type (Single / Double / Triple / Dormitory), sharing capacity, AC/Non-AC, attached bath, monthly rent, deposit, photos.
- Bed-level inventory for shared rooms: each bed is bookable individually (Available / Reserved / Occupied / Under Maintenance).
- **[v1.1]** `beds.status` is derived from bookings, tenancies and maintenance tasks by trigger. Owner manual block is recorded as a

maintenance task with `blocks_availability = true`.

7C. Search & Discovery (Student)

- Filters: city/area, near college (radius, using the `colleges` table), price range, room type, sharing, gender type, AC, food, facilities, availability date.
- Sort: price, distance, rating, newest. List view + map view.
- Property detail page: photos, rooms with live availability, rent/deposit, facilities, rules, reviews, location map.
- Wishlist/shortlist and side-by-side compare (up to 3 properties).

7D. Booking Module

- Flow: Student selects room/bed + move-in date + duration → submits booking request (optionally with token/advance amount) → Owner Approve / Reject within SLA (default 24h) → on approval, advance/deposit is paid (online in later phase; **recorded offline by the owner in v1**) → booking Confirmed → check-in → tenancy Active.
- **[v1.1] Atomic reservation:** on request, `create_booking_request()` locks the bed row (`SELECT ... FOR UPDATE`), verifies the bed, room and property are bookable, and inserts the booking. The partial unique index on active bookings per bed is the final backstop.
- **[v1.1] Two separate clocks:** `owner_response_due_at` (owner SLA, default 24h) and `payment_due_at` (student/owner payment window, default 24h, set on approval). `hold_expires_at` is the later of the two and is maintained only by database functions. An idempotent scheduled job expires overdue holds.
- Booking modes (configurable per property): Instant Book or Owner Approval Required.
- Cancellation rules per property: refund policy (full / partial / none by days before move-in), applied automatically to refund calculation.
- Visit scheduling (optional): student requests a property visit slot; owner confirms.

7E. Tenant Management (Owner)

- Tenant record: student profile link, room/bed, check-in date, expected check-out, rent amount, deposit paid, ID proof (access-controlled and logged), guardian contact, agreement PDF, status (Active / Notice Period / Checked-out / Evicted).
- Actions: check-in, room change (transfer), give/accept notice, check-out with deposit settlement (entries in `deposit_ledger`).
- Tenant history retained per property.

7F. Rent & Payment Module

- Monthly rent ledger: on tenancy start, system auto-generates rent dues each month (`rent_dues`) on a configured due day. Generation is idempotent (unique per tenancy and month). **[v1.1]**
- Payments: offline (cash/bank transfer/UPI) logged by owner with reference number (student confirmation optional); online (Razorpay: UPI/card/netbanking) in a later phase.
- Statuses: Due / Paid / Partially Paid / Overdue / Waived.
- Late fee: configurable per property (fixed or per-day) applied automatically after grace period (deferred in MVP).
- Extras: electricity/maintenance/other charges can be added to a month's due with description.
- Auto-generated PDF rent receipt for every payment; emailed to student.
- Owner view: collected vs pending per month, overdue list, tenant-wise ledger; Student view: dues, history, downloadable receipts.
- Reminders: before due date, on due date, and overdue (see Section 8).
- **[v1.1]** Security deposit ledger: `deposit_ledger` rows for collected, deduction and refund.

7F-1. Proration & Settlement Rules [v1.1]

- **Move-in mid-month:** first due = `monthly_rent × remaining_days / days_in_month` (calendar month, IST). Alternatively (owner setting) the rent cycle is anchored to the move-in date; one mode per property.
- **Notice period:** last stay date = notice date + `notice_period_days`. Final due is prorated to that date. The bed becomes available the next day.
- **Check-out:** `refund = deposit_collected − sum(deductions)`, all recorded as `deposit_ledger` rows; the refund flows through `refunds`.
- **Rounding:** half-up to 2 decimals, applied once per due.

7G. Complaint & Maintenance Module

- Student raises complaint: category (Electrical, Plumbing, Cleaning, WiFi, Furniture, Food, Security, Noise/Roommate, Other), description, photos, priority.
- Lifecycle: Open → Acknowledged → In Progress → Resolved → Closed (or Reopened by student within N days).
- Owner/Warden: assign to staff/vendor, add internal notes, set expected resolution date, upload resolution proof.
- SLA timers and auto-escalation to owner (from warden) and to Admin if unresolved beyond threshold.
- Student rates resolution (1-5) on closure.
- Owners can also log maintenance tasks on a room/property directly (preventive maintenance), which can mark beds Under Maintenance.

7H. Notices & Announcements

- Owner posts property-wide notices (water shutdown, rule changes, events); tenants notified via email/in-app.

7I. Reviews & Ratings

- Only students with a completed/active tenancy can review; one review per tenancy.
- Owner may respond publicly; Admin can remove abusive reviews (logged).

7J. Dashboards & Analytics

- Owner: occupancy rate, vacant beds, monthly collection vs pending, overdue tenants, open complaints, upcoming check-outs, booking requests pending.
- Student: current booking/tenancy, next rent due, open complaints.
- Admin: total users/properties/bookings, GMV, pending verifications, dispute count, city-wise stats.

7K. Global Search & Filters

- Owners: search tenants, bookings, payments, complaints by name/room/status/date. Admin: search across all entities.

8. Communication Templates

All emails/SMS/in-app messages are generated from a versioned template system.

Template	Trigger	Sent to
Welcome / Email Verification	Signup	Student / Owner
Owner Approval / Rejection Notice	Admin decision	Owner
Booking Request Received	Student submits request	Student, Owner
Booking Approved / Rejected	Owner decision	Student
Booking Payment Reminder	Approved but unpaid, before hold expiry	Student
Booking Confirmed	Advance recorded / payment success	Student, Owner
Booking Cancelled / Expired	Cancel or hold timeout	Student, Owner
Check-in Instructions	Before move-in date	Student
Rent Due Reminder	X days before due date	Student
Rent Overdue Notice	After due date + grace	Student (copy Owner)
Payment Receipt	Payment recorded/success	Student
Complaint Received / Status Update / Resolved	Complaint lifecycle events	Student, Owner
Notice / Announcement	Owner posts notice	Tenants
Notice Period Acknowledgement	Tenant/Owner gives notice	Both
Check-out & Deposit Settlement	Check-out completed	Student
Review Request	After check-out or N days of stay	Student
Guardian Consent Request **[v1.1]**	Minor student signs up	Guardian

9. Cancellation, Notice & Refund Handling

- Booking cancellation by student before check-in: refund computed automatically from the property's cancellation policy; recorded in `refunds` (gateway API call when online payments are enabled).
- Owner cancellation/rejection after approval: 100% refund + flag on owner reliability score.
- Notice period: either party may give notice; system computes last stay date and final rent (Section 7F-1); room auto-marked Available from that date.
- Eviction is Owner-initiated with a mandatory reason and Admin-visible log; deposit settlement follows the same flow.
- No-show: if student doesn't check in within N days of move-in date, booking is flagged; owner decides forfeit/refund per policy.
- Reply/chat parsing is out of scope for v1; offline communications are manually logged.

10. Notification & Deliverability Handling

- Email provider configured with bounce/complaint webhooks → `email_bounced = true` on user record; Admin/Owner alerted so critical messages (payment, notices) don't silently fail.
- Suppression list for hard-bounced addresses until corrected.
- In-app notifications for all events; email/SMS toggles per user.
- Aggregate bounce and failure rates feed the monitoring dashboard.

11. End-to-End Status Pipelines

11.1 Booking Pipeline

`requested` → `approved` / `rejected` → `payment_pending` → `confirmed` → `checked_in` (tenancy active) → `notice` → `checked_out` → `deposit_settled` → `closed`

`cancelled`, `expired` and `no_show` can branch off from the first four stages.

MVP variant [v1.1]: `requested` → owner approves → `payment_pending` → owner records the **offline advance** (`POST /payments/offline`, type `booking_advance`, with reference number) → `confirmed` → check-in → tenancy `active`. The online path later plugs into the same `payment_pending → confirmed` transition; only the payment source changes.

11.2 Rent Due Pipeline

Generated → Due → (Partially Paid) → Paid / Overdue → Waived (Owner only, logged)

11.3 Complaint Pipeline

Open → Acknowledged → In Progress → Resolved → Closed (or Reopened)

11.4 Owner/Property Verification

Submitted → Under Review → Approved / Rejected → (Suspended)

12. Data Model

Conventions **[v1.1]**: money is `numeric(12,2)` INR; all timestamps are `timestamptz`; due dates computed in `Asia/Kolkata`; primary keys are UUID.

12.1 users

`user_id` (PK UUID, **FK → auth.users(id)**), `role` (enum: student, owner, admin, warden), `name`, `email`, `phone`, `gender` (male/female/other/prefer_not_to_say), `dob`, `is_minor` (generated from dob), `email_verified`, `phone_verified`, `email_bounced` (bool), `consent_version`, `consented_at`, `mfa_enabled`, `status` (Active/Suspended), `created_at`, `retention_expires_at`

12.2 student_profiles

`student_id` (PK, FK users), `college`, `course`, `guardian_name`, `guardian_phone`, `guardian_consent_at`, `guardian_consent_version`, `guardian_verified` (bool), `id_proof_url` (private; image only, no ID number), `home_city`, `emergency_contact`

12.3 owner_profiles

`owner_id` (PK, FK users), `business_name`, `kyc_document_url`, `verification_status` (enum), `verified_by`, `verified_at`, `rejection_reason`, `payout_account_ref`

12.4 properties

`property_id` (PK), `owner_id` (FK), `name`, `type` (enum: boys_hostel, girls_hostel, coed, pg), `address`, `city`, `area`, `latitude`, `longitude`, `description`, `house_rules`, `facilities` (jsonb / text[]), `photos` (jsonb), `security_deposit_default`, `notice_period_days`, `lock_in_months`, `booking_mode` (enum: instant, approval), `cancellation_policy` (jsonb), `late_fee_config` (jsonb), `rent_due_day`, `rent_cycle_mode` (calendar / move_in_anchored), `status` (Draft/Pending/Approved/Suspended/Inactive), `avg_rating`, `created_at`

12.5 rooms

`room_id` (PK), `property_id` (FK), `room_number`, `floor`, `room_type` (enum: single, double, triple, dorm), `capacity`, `is_ac`, `attached_bath`, `monthly_rent`, `deposit`, `photos` (jsonb), `status` (Active/Inactive). **Unique (property_id, room_number).**

12.6 beds

`bed_id` (PK), `room_id` (FK), `bed_label`, `status` (enum: available, reserved, occupied, maintenance; **derived cache, trigger-maintained**), `available_from` (date), `reserved_until` (timestamptz, nullable)

12.7 bookings

`booking_id` (PK), `student_id` (FK), `property_id` (FK), `room_id` (FK), `bed_id` (FK), `move_in_date`, `duration_months`, `rent_amount`, `deposit_amount`, `advance_paid`, `status` (booking_status enum, see 11.1), `requested_at`, `responded_at`, `responded_by`, `rejection_reason`, **`owner_response_due_at`, `payment_due_at`**, `hold_expires_at`, `cancelled_at`, `cancel_reason`

Refund amounts and status live in `refunds` (12.21).

12.8 tenancies

`tenancy_id` (PK), `booking_id` (FK), `student_id` (FK), `property_id` (FK), `bed_id` (FK), `check_in_date`, `expected_checkout_date`, `actual_checkout_date`, `rent_amount`, `deposit_paid`, `agreement_url`, `status` (tenancy_status: active/notice/checked_out/evicted), `notice_given_on`, `notice_given_by`

Deposit deductions and refunds live in `deposit_ledger` (12.22).

12.9 rent_dues

`due_id` (PK), `tenancy_id` (FK), `month` (date), `base_amount`, `extra_charges` (jsonb), `late_fee`, `total_amount`, `amount_paid`, `due_date`, `status` (Due/Partial/Paid/Overdue/Waived), `waived_by`, `waive_reason`. **Unique (tenancy_id, month).**

12.10 payments

`payment_id` (PK), `due_id` (FK, nullable), `booking_id` (FK, nullable), `tenancy_id` (FK, **nullable**), `payer_id` (FK), `amount`, `type` (enum: booking_advance, deposit, rent, late_fee, refund), `mode` (enum: online, cash, bank_transfer, upi_offline), `gateway_order_id` (**unique**), `gateway_payment_id` (unique, idempotency), `reference_no`, `status` (Pending/Success/Failed/Refunded), `recorded_by`, `paid_at`, `receipt_url`. **CHECK: at least one of `due_id`, `booking_id`, `tenancy_id` is set.**

12.11 complaints

`complaint_id` (PK), `tenancy_id` (FK), `student_id` (FK), `property_id` (FK), `category`, `description`, `photos` (jsonb), `priority`, `status`, `assigned_to`, `expected_resolution_date`, `resolution_notes`, `resolution_proof` (jsonb), `student_rating`, `escalated` (bool), `created_at`, `resolved_at`, `closed_at`

12.12 complaint_updates

`update_id` (PK), `complaint_id` (FK), `author_id`, `visibility` (public/internal), `message`, `status_change`, `created_at`

12.13 maintenance_tasks

`task_id` (PK), `property_id` (FK), `room_id` (FK, nullable), `bed_id` (FK, nullable), `title`, `description`, `assigned_vendor`, `cost`, `status`, `scheduled_on`, `completed_on`, `blocks_availability` (bool)

12.14 notices

`notice_id` (PK), `property_id` (FK), `title`, `body`, `posted_by`, `posted_at`, `expires_at`

12.15 reviews

`review_id` (PK), `tenancy_id` (FK, unique), `property_id` (FK), `student_id` (FK), `rating`, `comment`, `owner_reply`, `status` (Visible/Hidden), `created_at`

12.16 wishlists

`user_id` (FK), `property_id` (FK), `added_at`

12.17 visit_requests

`visit_id` (PK), `student_id` (FK), `property_id` (FK), `requested_slot`, `status` (Requested/Confirmed/Declined/Done)

12.18 notification_templates & notification_logs

Templates: `template_id`, `name`, `channel` (email/sms/in_app), `version`, `body`, `is_active`. Logs: `log_id`, `user_id`, `template_id`, `recipient`, `subject`, `status`, `sent_at`, `error`, `retry_count`, `bounce_type`

12.19 audit_logs

`log_id` (PK), `user_id`, `role`, `action`, `entity_type`, `entity_id`, `timestamp`, `previous_value`, `new_value`. Append-only, no UPDATE/DELETE grants at DB role level. **[v1.1]** Values store changed field names, IDs and non-PII values only; PII fields (name, email, phone, ID URLs, guardian data) are logged as `"[redacted]"` with a hash of the old value if change detection is needed, so erasure never requires editing the log.

12.20 data_access_requests

`request_id` (PK), `user_id`, `request_type` (Access/Erasure), `status` (Pending/Fulfilled), `requested_at`, `fulfilled_at`, `fulfilled_by`, `export_file_url`

12.21 refunds [v1.1 new]

`refund_id` (PK), `payment_id` (FK), `booking_id` (FK, nullable), `tenancy_id` (FK, nullable), `amount` (> 0), `reason`, `gateway_refund_id` (unique, nullable), `status` (pending/processed/failed), `created_at`

12.22 deposit_ledger [v1.1 new]

`entry_id` (PK), `tenancy_id` (FK), `entry_type` (collected/deduction/refund), `amount` (> 0), `reason`, `created_by`, `created_at`

12.23 property_staff [v1.1 new]

`property_id` (FK), `user_id` (FK), `staff_role` (warden/manager), `invited_by`, `created_at`. PK (property_id, user_id). `users.role = warden` is a coarse flag; property access comes from this table.

12.24 colleges [v1.1 new]

`college_id` (PK), `name`, `city`, `latitude`, `longitude`. Unique (name, city). Seeded for the launch city; feeds the "near college" filter.

12.25 webhook_events [v1.1 new]

`event_id` (PK, gateway event id), `provider`, `payload` (jsonb), `received_at`, `processed_at`, `error`

Key Constraints & Indexes

- **Double-booking guard (database level):**
  - `unique (bed_id)` on `bookings` where status in (`requested`, `approved`, `payment_pending`, `confirmed`, `checked_in`, `notice`).
  - `unique (bed_id)` on `tenancies` where status in (`active`, `notice`).
- Unique: `(property_id, room_number)`, `rent_dues(tenancy_id, month)`, `payments(gateway_order_id)`, `payments(gateway_payment_id)`, `refunds(gateway_refund_id)`.
- Indexes: `properties(city, area, status)`, `rooms(property_id, monthly_rent)`, `beds(status)`, `rent_dues(status, due_date)`, `bookings(status, hold_expires_at)`.
- Row Level Security (Supabase RLS): students see own rows; owners see rows for properties they own; wardens see rows for properties in `property_staff`; admin sees all. Applied through JWT-scoped sessions (Section 4A).

12.26 Reference SQL [v1.1]

```
create type booking_status as enum (
  'requested','approved','payment_pending','confirmed','checked_in',
  'notice','checked_out','deposit_settled','closed',
  'rejected','cancelled','expired','no_show');
create type tenancy_status as enum ('active','notice','checked_out','evicted');

alter table users
  add constraint users_auth_fk foreign key (user_id) references auth.users(id),
  add column dob date,
  add column gender text check (gender in ('male','female','other','prefer_not_to_say')),
  add column is_minor boolean generated always as
    (dob is not null and dob > (current_date - interval '18 years')) stored;

alter table student_profiles
  add column guardian_consent_at timestamptz,
  add column guardian_consent_version text,
  add column guardian_verified boolean not null default false;

alter table rooms add constraint uq_room_number unique (property_id, room_number);

alter table bookings
  add column owner_response_due_at timestamptz,
  add column payment_due_at timestamptz;

create unique index uq_bed_active_booking on bookings(bed_id)
  where status in ('requested','approved','payment_pending','confirmed','checked_in','notice');
create unique index uq_bed_active_tenancy on tenancies(bed_id)
  where status in ('active','notice');

alter table rent_dues add constraint uq_due_month unique (tenancy_id, month);

alter table payments alter column tenancy_id drop not null;
alter table payments add constraint chk_payment_target
  check (num_nonnulls(due_id, booking_id, tenancy_id) >= 1);
alter table payments add constraint uq_gateway_order unique (gateway_order_id);

create table webhook_events (
  event_id text primary key, provider text not null, payload jsonb not null,
  received_at timestamptz not null default now(), processed_at timestamptz, error text);

create table property_staff (
  property_id uuid references properties(property_id) on delete cascade,
  user_id uuid references users(user_id) on delete cascade,
  staff_role text not null check (staff_role in ('warden','manager')),
  invited_by uuid references users(user_id),
  created_at timestamptz not null default now(),
  primary key (property_id, user_id));

create table colleges (
  college_id uuid primary key default gen_random_uuid(),
  name text not null, city text not null,
  latitude double precision not null, longitude double precision not null,
  unique (name, city));

create table refunds (
  refund_id uuid primary key default gen_random_uuid(),
  payment_id uuid not null references payments(payment_id),
  booking_id uuid references bookings(booking_id),
  tenancy_id uuid references tenancies(tenancy_id),
  amount numeric(12,2) not null check (amount > 0),
  reason text, gateway_refund_id text unique,
  status text not null check (status in ('pending','processed','failed')),
  created_at timestamptz not null default now());

create table deposit_ledger (
```

```
  entry_id uuid primary key default gen_random_uuid(),
  tenancy_id uuid not null references tenancies(tenancy_id),
  entry_type text not null check (entry_type in ('collected','deduction','refund')),
  amount numeric(12,2) not null check (amount > 0),
  reason text, created_by uuid references users(user_id),
  created_at timestamptz not null default now());

create function sync_bed_status() returns trigger language plpgsql as $$
begin
  update beds set status = case
    when exists (select 1 from tenancies t where t.bed_id = new.bed_id
                 and t.status in ('active','notice')) then 'occupied'
    when exists (select 1 from bookings b where b.bed_id = new.bed_id
                 and b.status in ('requested','approved','payment_pending','confirmed')) then 'reserved'
    when exists (select 1 from maintenance_tasks m where m.bed_id = new.bed_id
                 and m.blocks_availability and m.status <> 'done') then 'maintenance'
    else 'available' end
  where bed_id = new.bed_id;
  return new;
end $$;
create trigger trg_bookings_sync after insert or update of status on bookings
  for each row execute function sync_bed_status();
create trigger trg_tenancies_sync after insert or update of status on tenancies
  for each row execute function sync_bed_status();

create function create_booking_request(
  p_student uuid, p_bed uuid, p_move_in date, p_months int)
returns uuid language plpgsql as $$
declare
  v_bed beds%rowtype; v_room rooms%rowtype; v_prop properties%rowtype;
  v_id uuid; v_owner_due timestamptz := now() + interval '24 hours';
begin
  select * into v_bed from beds where bed_id = p_bed for update;
  if not found then raise exception 'BED_NOT_FOUND'; end if;
  select * into v_room from rooms where room_id = v_bed.room_id;
  select * into v_prop from properties where property_id = v_room.property_id;
  if v_prop.status <> 'Approved' or v_room.status <> 'Active' then
    raise exception 'NOT_BOOKABLE'; end if;
  if v_bed.status <> 'available' then raise exception 'BED_UNAVAILABLE'; end if;
  insert into bookings(student_id, property_id, room_id, bed_id, move_in_date,
    duration_months, rent_amount, deposit_amount, status, requested_at,
    owner_response_due_at, hold_expires_at)
  values (p_student, v_prop.property_id, v_room.room_id, p_bed, p_move_in, p_months,
    v_room.monthly_rent, v_room.deposit,
    case when v_prop.booking_mode = 'instant' then 'payment_pending' else 'requested' end,
    now(), v_owner_due, v_owner_due)
  returning booking_id into v_id;
  return v_id;
end $$;
```

On owner approval: `status = 'payment_pending'`, `payment_due_at = now() + interval '24 hours'`, `hold_expires_at = payment_due_at`.
Expiry job (idempotent): `update bookings set status = 'expired' where status in ('requested','approved','payment_pending') and hold_expires_at < now();`

13. API Surface (indicative)

Method	Endpoint	Purpose
POST	/auth/signup, /auth/login	Register / login
POST	/auth/password-reset, /auth/phone-otp	Password reset, phone OTP **[v1.1]**
POST	/auth/mfa/enroll, /auth/mfa/verify	Admin MFA
GET/PUT	/users/me	Profile
POST	/students/guardian-consent	Record/verify guardian consent **[v1.1]**
POST	/owners/kyc	Submit owner KYC
POST	/admin/owners/{id}/verify	Approve/reject owner
POST	/admin/properties/{id}/verify	Approve/reject property
GET	/properties	Search/filter (city, price, type, facilities, distance, availability)
GET	/properties/{id}	Property detail with rooms and availability
POST / PUT / DELETE	/properties, /properties/{id}	Owner: create/update/deactivate
POST / PUT	/properties/{id}/rooms, /rooms/{id}	Owner: room management
POST / DELETE	/properties/{id}/staff	Invite/remove warden or manager **[v1.1]**
PATCH	/beds/{id}/status	Block/unblock bed (creates maintenance block)
POST / DELETE	/wishlist, /wishlist/{property_id}	Shortlist
POST / PATCH	/visit-requests, /visit-requests/{id}	Visit scheduling
POST	/bookings	Student: create booking request (atomic bed hold)
POST	/bookings/{id}/respond	Owner: approve/reject

POST	/bookings/{id}/pay	Create gateway order (later phase)
POST	/webhooks/payment	Razorpay webhook (signature verified, idempotent; later phase)
POST	/bookings/{id}/cancel	Cancel + auto refund calc
GET	/bookings	List (role-scoped)
POST	/tenancies/{booking_id}/check-in	Owner: check-in
POST	/tenancies/{id}/room-transfer	Move tenant to another bed
POST	/tenancies/{id}/notice	Give notice
POST	/tenancies/{id}/check-out	Check-out + deposit settlement
GET	/tenancies	Tenant list (owner) / own tenancy (student)
GET	/rent-dues	Ledger (role-scoped, filter by month/status)
POST	/rent-dues/{id}/charges	Add extra charge
POST	/rent-dues/{id}/pay	Online payment order (later phase)
POST	/payments/offline	Owner: record cash/bank/UPI payment (incl. booking advance)
POST	/rent-dues/{id}/waive	Owner: waive (logged)
GET	/payments/{id}/receipt	Download PDF receipt
POST	/complaints	Student: raise complaint
PATCH	/complaints/{id}	Update status/assign/resolve
POST	/complaints/{id}/updates	Add comment/internal note
POST	/complaints/{id}/reopen, /rate	Reopen / rate
POST / GET	/maintenance-tasks	Owner maintenance log
POST / GET	/notices	Announcements
POST	/reviews, /reviews/{id}/reply	Reviews
GET	/dashboard/owner, /dashboard/student, /dashboard/admin	Aggregated stats
GET	/audit-logs	Activity log (admin)
GET	/export/{resource}	CSV/Excel export
POST	/data-access-requests, /data-access-requests/{id}/fulfill	Access/erasure **[v1.1 typo fixed]**

14. Security

- Role-based access (student / owner / warden / admin) enforced in the backend **and** Supabase RLS; FastAPI sessions carry the caller's JWT so RLS applies (Section 4A). **[v1.1]**
- Mandatory MFA for Admin; optional for owners.
- JWT access tokens expire in 1 hour; refresh tokens 14 days or on password change; admin can force-revoke sessions.
- Per-user and per-IP rate limiting; extra limits on search, signup, and export endpoints.
- **Payments (when online payments are enabled) [v1.1]:**
  1. Verify signature on the raw body.
  2. `insert into webhook_events(...) on conflict do nothing`; if nothing inserted, return 200 (replay).
  3. Look up the payment by `gateway_order_id`; validate amount and currency against the server-side order; set `gateway_payment_id` and `status = Success`; advance the booking/due in the same transaction.
  4. Mark `processed_at`. On error, record `error` and return 5xx so the gateway retries.
  5. A reconciliation job re-fetches gateway records for any Pending payment older than 30 minutes.
  6. Never trust client amounts. No card data stored.
- ID proofs, agreements, receipts in private buckets (signed URLs only); room photos in public bucket with size/type validation. **[v1.1]** Store the ID image only; never store or log the ID number. Owners access ID proofs only for their own confirmed tenants and approved applicants; every access is written to `audit_logs`.
- Passwords hashed (bcrypt/argon2); input validation/sanitisation everywhere; file type/size checks on uploads.
- Owner sees student PII only for their own applicants/tenants, limited as in Section 3.
- Contact details of owner/student revealed only after booking approval.
- No PII in logs beyond what the audit trail requires (which itself redacts PII, Section 12.19).
- Public endpoints protected by CAPTCHA + rate limiting.

15. Fairness, Trust & Safety

- Reviews verified to actual tenancies; abusive content moderation by Admin.
- Owner reliability signals: response time, approval rate, cancellation rate, complaint resolution time (shown as badges, factual only).

- Search ranking uses only objective fields (price, distance, rating, availability); paid promotion, if ever added, must be clearly labelled.
- Gender-specific properties (girls/boys hostel) enforced at booking using the student profile gender; co-ed and PG accept all values. **[v1.1]** Students who chose "other" or "prefer not to say" applying to a single-gender hostel go to Admin-visible manual review rather than an automatic block.
- Reports/flagging on listings and users; Admin can suspend with logged reason.

16. Data Retention & Compliance

- Primary framework: India's Digital Personal Data Protection Act, 2023 (DPDP Act).
- Consent captured at signup with `consent_version` and `consented_at`; versioned consent text.
- **[v1.1] Minors:** DOB collected at signup. Users under 18 require verifiable guardian consent (`guardian_consent_at`, `guardian_consent_version`, guardian OTP-verified) before booking. Guardian consent text is versioned.
- Configurable retention per record type (e.g., ID proofs purged N months after check-out; inactive accounts flagged after N months); enforced via `retention_expires_at` + scheduled purge job.
- Right to erasure: Admin-mediated; personal fields removed, anonymised financial/booking records retained where legally required (payment/tax records). **[v1.1]** Audit logs hold no raw PII, so erasure does not touch the append-only log.
- Right to access: `data_access_requests` record → structured JSON/PDF export of all data held.
- All retention/erasure/access actions logged in the append-only audit log.
- Retention window and financial-record retention = legal/stakeholder decision (Section 27).

17. Reporting & Data Export

- CSV/Excel export: Bookings, Tenants, Rent Ledger, Payments, Complaints, Maintenance, Occupancy, Deposit Ledger; respecting active filters.
- Owner reports: monthly collection report, occupancy report, overdue report, deposit ledger, complaint SLA report.
- Admin reports: platform GMV, city-wise supply/demand, owner performance, verification turnaround.
- Scheduled exports (e.g., monthly collection summary emailed to owner) as optional configuration.

18. Performance Targets

Operation	Target
Property search (with filters, first page)	< 1.5 seconds
Property detail page load	< 2 seconds
Booking creation (atomic bed hold)	< 2 seconds
Payment order creation	< 3 seconds
Webhook processing (payment → status update)	< 5 seconds
PDF receipt generation	< 5 seconds
Notification enqueue → provider handoff	< 15 seconds
Monthly rent-due generation (1,000 tenancies)	< 2 minutes
Dashboard load (cached/materialised view)	< 2 seconds
CSV/Excel export (up to 5,000 rows)	< 30 seconds

Starting targets for design purposes; validate against real infra before Phase 1 sign-off.

19. Error Handling & Resilience

- Every operation (booking, payment, webhook, PDF, notification, scheduled job) has defined error states and a Retry action where recoverable.
- Payment reconciliation job (online phase): periodically compares gateway records with payments to catch missed webhooks.
- Booking hold expiry, rent generation, and overdue flags are idempotent scheduled jobs (rent generation protected by the unique `(tenancy_id, month)` constraint).
- Failures surfaced to Owner/Admin via UI + notifications, never silent.
- Notification failures logged with retry count and last error.

20. Monitoring & Observability

- Sentry for frontend and backend errors with alerting on elevated error rates.
- Uptime monitoring on public search, login, and payment webhook endpoints.
- Notification deliverability monitoring (bounce/failure rates).
- Background job health (Celery queue depth, stuck-job alerts).
- Payment success/failure rate and webhook lag dashboard (online phase).

21. Environments & Release Process

- Two isolated environments: staging and production, each with its own Supabase project and separate email/payment credentials (Razorpay test mode in staging; staging never messages real users).
- Branching: feature branches → PR → `develop` (auto-deploy staging) → tagged release → `main` (deploy production).
- Template and pricing/fee-logic changes validated in staging against seeded, anonymised data before promotion.
- Backup & DR: Supabase daily backups retained 30 days; weekly export to cold storage; RTO 4 hours, RPO 24 hours, validated via restore drill before go-live.

22. QA & Testing Strategy

- Unit tests on backend services (booking hold/expiry, availability logic, rent-due generation, late fee, refund, proration, status transitions), minimum coverage threshold in CI (e.g., 70%).
- **[v1.1] Concurrency:** 20 parallel `create_booking_request` calls on one bed → exactly 1 succeeds, 19 get `BED_UNAVAILABLE`. Repeat with the row lock disabled to confirm the partial unique index alone still blocks duplicates.
- **[v1.1] Status sync:** every booking/tenancy/maintenance transition leaves `beds.status` consistent (property-based test over random transition sequences).
- **[v1.1] Idempotency:** replay the same webhook 5 times → 1 payment update, 1 receipt. Rent generation run twice for a month → no duplicate `rent_dues`.
- Integration tests on full flows: signup → search → booking → approval → payment/advance → check-in → rent cycle → complaint → notice → check-out → deposit settlement.
- Payment tests using gateway sandbox, including duplicate/replayed webhooks and failed payments (online phase).
- **[v1.1] RLS/permission tests through the API path** with real user JWTs (not the service key): student A cannot read student B's data, owner A cannot read owner B's data, warden sees only assigned property.
- **[v1.1] Minor gate:** booking rejected for a minor without guardian consent.
- **[v1.1] Proration/refund:** table-driven tests for mid-month move-in, notice, and deposit deductions.
- UAT sign-off per phase with documented checklist in staging.
- Manual QA: mobile responsiveness, email rendering (Gmail/Outlook), PDF rendering before each release.

23. Loading States & UX

- Loading indicators for search, booking, payment, PDF generation, uploads, exports; triggering buttons disabled during processing to prevent duplicate submissions (especially payment).
- Skeleton loaders on property lists; optimistic UI only for non-financial actions.
- Clear empty states ("No rooms match your filters") and error states with retry.

24. Responsiveness

- Students are mostly on phones: mobile-first design for search, booking, payment, and complaints.
- Owner portal: full data tables on desktop; tables collapse to cards on tablet/mobile; all forms, filters, and dashboards fully usable at small breakpoints.
- Image optimisation (thumbnails, lazy loading) for photo-heavy property pages.

25. System Navigation Structure

Student Portal: Home / Search (List / Map) · Property Detail · Wishlist & Compare · My Bookings · My Tenancy (details, agreement) · Rent & Payments (dues, history, receipts) · Complaints · Notices · Reviews · Profile & Documents · Notifications

Owner Portal: Dashboard · Properties · Rooms & Beds · Booking Requests · Tenants · Rent Ledger (Dues / Payments / Overdue) · Deposits · Complaints & Maintenance · Notices · Reviews · Staff (Wardens) · Reports & Export · Notifications · Settings (policies, late fees, cancellation rules)

Admin Panel: Dashboard · Owner Verification · Property Verification · Users · Bookings & Payments · Disputes / Reports · Reviews Moderation · Notification Tracking (Bounce/Suppression) · Reports & Export · Data Requests (Access / Erasure) · Audit Logs · Settings (Users & Roles, MFA, Sessions)

26. Implementation Phasing (Suggested)

Phase	Scope
1	Supabase schema + RLS (staging & production) with JWT-scoped sessions; `auth.users` FK; Auth with roles; student & owner registration; DOB/gender/minor fields; consent capture; CAPTCHA/rate limiting; admin MFA; `property_staff` **[v1.1]**
2	Owner KYC + admin verification; Property CRUD; Room & Bed management; photo uploads; `colleges` seed for launch city **[v1.1]**
3	Search & filters, property detail page, map view, wishlist/compare
4	Booking module: request, atomic bed hold (`create_booking_request`), both partial unique indexes, bed-sync triggers, owner approve/reject, expiry job, cancellation rules; write the concurrency test first **[v1.1]**

5	Payment integration (online, deferred past MVP): Razorpay orders, `webhook_events`, unique `gateway_order_id`, reconciliation job, receipts (PDF), `refunds` **[v1.1]**
6	Tenancy: check-in, agreement PDF, room transfer, notice, check-out, `deposit_ledger` settlement **[v1.1]**
7	Rent ledger: monthly due generation, proration, late fees, extra charges, offline payment logging, reminders, overdue flags
8	Complaint & maintenance module: lifecycle, assignment, SLA/escalation, ratings
9	Notices, reviews & ratings, notifications/templates, deliverability & bounce handling, append-only PII-safe audit log
10	Dashboards & analytics (owner/student/admin), reporting/export, data retention automation, access/erasure workflow
11	Security hardening, load/concurrency testing, full QA (unit/integration/UAT), backup/DR drill, go-live

27. Open Decisions for Stakeholders

Decision	Status
Payment model	**Decided for v1: record-only** (owner collects; platform records). Payout/commission via Razorpay Route is an open Phase 5 question.
Commission/fee model: free listing, per-booking fee, or subscription	Open (Phase 5)
Booking default: Instant Book or Owner Approval Required	Open (suggest Owner Approval)
Booking hold window length (default 24h) and token/advance amount (fixed % or flat)	Open
Cancellation/refund policy: platform-wide default vs per-property only	Open
Late-fee rules: allowed, max cap, grace period	Open
Warden/Manager role at launch or deferred (schema ready either way)	Open
Bed-level vs room-level booking at launch	Suggest room-level (one bed per room), bed-level enabled later without migration
ID proof mandatory for all students before booking	Open
Retention windows for ID proofs, payment records, inactive accounts (legal sign-off on DPDP language)	Open
Guardian consent method for minors (OTP vs document)	Open **[v1.1]**
Notification channels at launch: email only, or email + SMS/WhatsApp	Open
City/region launch scope	Open
Rental agreement: platform template only, or owner-uploaded custom agreements	Open
Review moderation: pre-approval or post-moderation	Open
Confirm performance targets in Section 18 with engineering before Phase 1 sign-off	Open

28. Proposed Enhancements: Future Candidates

Worth adding soon (high value, fits existing architecture)

- Roommate matching: optional preferences (sleep schedule, diet, study habits) to suggest compatible roommates for shared rooms.
- UPI AutoPay / recurring rent mandates to reduce overdue rent and manual reminders.
- WhatsApp reminders for rent due and complaint updates.
- Digital e-sign of rental agreement (Aadhaar eSign / DocuSign-style provider).

Medium-term additions

- In-app chat between student and owner with moderation and contact masking.
- Visitor management: pre-registered visitor log with gate approval.
- Mess/food module: menu, meal plans, attendance, monthly food billing.
- Electricity sub-metering: per-room meter readings feeding rent extras.
- Attendance/curfew tracking: check-in/out log for girls' hostels and guardian alerts.
- Referral program: student-refers-student credits.
- Duplicate "Merge" tooling for Admin.

Bigger, only if scale demands

- PostGIS radius search & recommendation ranking using search/booking behaviour.

- Multi-property chain support: organisation-level accounts with multiple managers and consolidated reporting.
- Native mobile apps (React Native) with push notifications.
- IoT/smart-lock or biometric access integration.
- Multi-language UI (Hindi, Gujarati, etc.).
- Accounting integration (Tally/Zoho Books) and GST invoicing for owners.

Recommended next step: start with Phases 1-4 as the MVP (auth, listings, search, booking) and add the offline rent ledger; spec the online payment flow (Phase 5) in isolation early, since the payout/commission decision shapes the data model.

29. Build Recommendations

1. **Cut the MVP hard.** Build auth, property/room listing, search, booking request with owner approval, and the offline rent ledger. Defer online payments, reviews, wardens, and maps.
2. **Record rent instead of collecting it (first release).** Owners log cash/UPI payments (including the booking advance) and the system generates receipts. Add Razorpay in a later phase.
3. **Start with room-level booking.** Keep the `beds` table but create one bed per room; `create_booking_request` already takes `bed_id`, so per-bed booking needs no migration later.
4. **Prevent double-booking in the database.** Use the row lock plus the partial unique indexes (Section 12). Do not rely on availability checks in application code. Write the concurrency test first.
5. **Set up RLS on day one**, exercised through JWT-scoped sessions. Write policies per table as it is created.
6. **Add Celery beat jobs early.** Three jobs keep data correct: monthly rent-due generation, release of expired booking holds, and overdue-rent flagging.
7. **Launch in one city.** Start near a few colleges, seed the `colleges` table, and add 5-10 real properties by talking to local owners.
8. **Add differentiators cheaply.** Roommate matching, distance from the student's college, and WhatsApp rent reminders are low effort and highly visible to students.

Suggested MVP Scope

Include in MVP	Defer
Auth + roles + RLS (JWT-scoped)	Online payments (Razorpay)
Owner KYC + admin approval	Reviews & ratings
Property, room, bed CRUD	Warden role
Search + filters + detail page	Map view, PostGIS radius search
Booking request + owner approval + hold expiry + offline advance	Instant Book, refunds automation
Tenancy check-in / check-out	Room transfer, deposit automation beyond the ledger
Offline rent ledger + PDF receipts	Late fees, extra charges
Complaints (basic lifecycle)	SLA escalation, vendor assignment
Guardian consent gate for minors	Duplicate "Merge"